Beginner Resources for K-Means in R

Agent: PURPLE ONEchat

[10/29/2026 01:31:17 +00:00] Jillyan M. Corrales:

I am a student in a 400-level archaeology class "Applications of Artificial Intelligence in Archaeology". For my final project I plan to use this method:

k-means

on this dataset:

Zaitceva, Olga, Vodyasov, Evgeny, and Vavulin, Mikhail. The Kokentau Ochre Rock Paintings Dataset. Version 2. Zenodo. 2026. https://doi.org/10.5281/zenodo.18665683.

to investigate this research question:

Are there spatial clusters in motifs or symbols painted onto rock surfaces in the Kokentau dataset?

I am a beginner in R. I have used the tidyverse and Quarto for only a few weeks, and I can import a CSV file and clean columns with dplyr. Please recommend:

Two short, accessible, introductory readings (each under 20 pages or 30 minutes) that explain this method in plain language. For each, give the full reference, a URL or DOI, and one sentence on why it is useful for a beginner.
Two free online beginner tutorials for using this method in R, preferably written in the tidyverse style. For each, give the URL, the approximate time to complete.
The three best-documented and most current R packages for this method. For each, give the package name, what it is best for, its CRAN or GitHub URL, the version and date of its most recent release as far as you know, and a link to an online walk-through of the core functionality of the package.
Only give me URLs and references that you are confident exist. If you are unsure about a link, version number or date, say so instead of guessing. Tell me your knowledge cutoff date so I know how current your package advice is.

Response:

I'll verify current package versions since you want up-to-date info, and my training data may be stale relative to October 2026.

Here's a curated set of resources for your k-means project on the Kokentau rock art dataset. One flag up front: k-means expects numeric, continuous coordinates/features, so for "spatial clusters in motifs/symbols" you'll likely be clustering on columns like x/y (or lat/long) panel coordinates, possibly combined with encoded motif-type variables — keep that in mind as you scope your approach.

Knowledge cutoff: My training data goes through early 2025. Everything below was verified against live web sources just now (dated October 2026), so the package version/date info reflects what's current today, not my training cutoff.

Two short plain-language readings

1. **James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in R* (2nd ed.), Section 12.4.1 "K-Means Clustering." Springer.** Free PDF at https://www.statlearning.com/. ~6 pages. This is the standard, widely-trusted beginner explanation of k-means with minimal math, written for exactly your stage (some stats background, new to the method) — no fabricated DOI needed since the whole book is a free, well-known download.

2. **Harris, N. (2014). "Visualizing K-Means Clustering."** https://www.naftaliharris.com/blog/visualizing-k-means-clustering/ — ~10–15 minutes [doc1]. An interactive, no-jargon walkthrough that lets you watch centroids move and reassign points step by step; it's the best "build intuition fast" resource I know of, and it also honestly covers where k-means fails (non-round clusters), which matters for archaeological spatial data that may be irregularly distributed.

Two free beginner R tutorials (tidyverse-adjacent)

1. **Kassambara, A. (2026). "Cluster Analysis in R: A Complete Step-by-Step Example."** https://www.datanovia.com/blog/cluster-analysis-in-r-example — ~20–25 minutes [doc2]. Walks through the *entire* realistic pipeline (scale → test clusterability → choose k via gap statistic → run `kmeans()` → validate with silhouette) on a real dataset, using `factoextra` and `cluster`. This maps directly onto what a reviewer will expect in a 400-level project: not just "run kmeans()" but justifying k and validating the result.

2. **GeeksforGeeks. "K-Means Clustering in R Programming."** https://www.geeksforgeeks.org/r-language/k-means-clustering-in-r-programming/ — ~15 minutes [doc3]. A more bare-bones, line-by-line intro (`kmeans()`, confusion matrix against known labels, `fviz_cluster()`). Good second pass if the Datanovia walkthrough moves too fast.

(Note: this one is base-R rather than tidyverse-piped; I didn't find a tidyverse-style beginner tutorial I could verify with full confidence today, so I'm flagging that gap rather than guessing a URL.)

Three best-documented, current R packages for k-means

Package	Best for	Repo	Latest version / date	Walkthrough

cluster	Core distance/partition methods (k-means complements: PAM, CLARA, gap statistic via `clusGap()`, silhouette validation)	https://cran.r-project.org/package=cluster	2.1.8.3, published 2026-07-30 [doc4] [doc5]	https://www.datanovia.com/blog/cluster-analysis-in-r-example [doc2] (uses `clusGap()` and `silhouette()` from this package)
factoextra	Visualizing clustering results as publication-ready ggplot2 figures (`fviz_cluster()`, `fviz_silhouette()`, `fviz_gap_stat()`)	https://cran.r-project.org/package=factoextra	2.2.0, published 2026-07-24 [doc6]	Same post above [doc2], or https://www.geeksforgeeks.org/r-language/k-means-clustering-in-r-programming/ [doc3]
tidyclust	A tidymodels-style unified interface to k-means (and other clustering), so it plugs into `recipes`/`workflows` the way you're already used to from tidyverse-adjacent modeling	https://github.com/tidymodels/tidyclust (CRAN: https://cran.r-project.org/package=tidyclust)	0.3.2, published 2026-06-28 [doc7]	https://tidyclust.tidymodels.org/ homepage walkthrough with `k_means()` [doc8]

Suggested path for your project: since you're new to R but comfortable with dplyr, I'd lean toward `tidyclust` for the actual clustering call (it feels like the modeling syntax you already know) paired with `factoextra` purely for mapping/visualizing which painted panels fall into which spatial cluster — that combination will read cleanly in a Quarto report. Happy to help you sketch the actual analysis pipeline (reading the Kokentau CSV, picking coordinate/motif columns, choosing k) once you're ready.

[10/29/2026 01:42:53 +00:00] Jillyan M. Corrales:

I am a student in a 400-level archaeology class "Applications of Artificial Intelligence in Archaeology". For my final project I plan to use this method:

Hu moments

on this dataset:

Zaitceva, Olga, Vodyasov, Evgeny, and Vavulin, Mikhail. The Kokentau Ochre Rock Paintings Dataset. Version 2. Zenodo. 2026. https://doi.org/10.5281/zenodo.18665683.

to investigate this research question:

Are there spatial clusters in motifs or symbols painted onto rock surfaces in the Kokentau dataset?

I am a beginner in R. I have used the tidyverse and Quarto for only a few weeks, and I can import a CSV file and clean columns with dplyr. Please recommend:

Two short, accessible, introductory readings (each under 20 pages or 30 minutes) that explain this method in plain language. For each, give the full reference, a URL or DOI, and one sentence on why it is useful for a beginner.
Two free online beginner tutorials for using this method in R, preferably written in the tidyverse style. For each, give the URL, the approximate time to complete.
The three best-documented and most current R packages for this method. For each, give the package name, what it is best for, its CRAN or GitHub URL, the version and date of its most recent release as far as you know, and a link to an online walk-through of the core functionality of the package.
Only give me URLs and references that you are confident exist. If you are unsure about a link, version number or date, say so instead of guessing. Tell me your knowledge cutoff date so I know how current your package advice is.

Response:

Here's what I found — and I want to flag upfront that Hu moments are a much more niche method in the R ecosystem than k-means, so the "readings/tutorials" landscape looks different than last time.

Knowledge cutoff: My training data ends in early 2025; everything below was verified against the live web just now (October 2026).

Two short plain-language readings

1. **Mallick, S. & Bapat, K. (2018, updated July 2026). "Shape Matching using Hu Moments (C++/ Python)." LearnOpenCV.** https://learnopencv.com/shape-matching-using-hu-moments-c-python/ — ~15–20 min read [doc1]. This is the clearest plain-language walkthrough of Hu moments I could verify: it builds up from raw moments → central moments → normalized moments → the 7 Hu invariants with minimal math, concrete worked examples (comparing letter shapes), and an honest "limitations" section about when Hu moments fail — directly useful for a beginner writing a methods section.

2. I was not able to verify a second short, beginner-friendly reading on Hu moments specifically with high confidence this session. Rather than guess a link, I'll flag the gap: the original source — **Hu, M.K. (1962). "Visual Pattern Recognition by Moment Invariants." IRE Transactions on Information Theory** — is the foundational paper but is not beginner-friendly. A reasonable substitute, which you should verify yourself before citing, is OpenCV's own "Image Moments" tutorial page in the official docs (docs.opencv.org) — I know this resource exists from general knowledge but didn't re-confirm its current URL/content this session, so treat it as unverified until you check it.

R tutorials for Hu moments — an important caveat

I could not find a tidyverse-style (or any) dedicated beginner R tutorial that walks through computing Hu moments step-by-step. This method lives almost entirely in Python/OpenCV tutorial culture; R coverage is sparse and mostly buried in package documentation rather than standalone tutorials. The closest things I can point you to with confidence:

1. **Rvision package function reference for `moments()`** — https://swarm-lab.github.io/Rvision/reference/moments.html — ~15 min [doc2]. Not a tutorial per se, but it's a runnable, documented example: reads an image, finds contours, computes raw/central/normalized moments on a real shape. It explicitly cross-references `huInvariants` (the actual Hu moment function) in its "See also" section.
2. **LearnOpenCV Hu moments post above** [doc1] — while it's Python code, the logic (threshold → `moments()` → `HuMoments()` → log-transform → compare shapes) maps almost line-for-line onto what you'd do with Rvision's `moments()`/`huInvariants()`, so it's worth working through as a translation exercise.

Three R packages for Hu moments — with honesty about fit

Because true "Hu's 7 invariant moments" are rare as a named function in R, here's the realistic landscape:

Package	Best for	Repo	Latest version/date	Walkthrough
Rvision	The actual thing you want: OpenCV-backed `moments()` + `huInvariants()` for true Hu invariant moments on contours/shapes	https://github.com/swarm-lab/Rvision (not on CRAN — GitHub/source install, requires OpenCV system library)	0.8.1 per GitHub docs page [doc3]; DESCRIPTION lists 0.8.0 / 2024-01-31 [doc4] — there's a slight version mismatch between sources, so check `packageVersion("Rvision")` after install	https://swarm-lab.github.io/Rvision/reference/moments.html [doc2]
EBImage (Bioconductor)	A well-documented, easy-to-install *approximation*: `computeFeatures.moment()` gives moment-derived shape descriptors (centroid, major axis, eccentricity, angle) that are translation/rotation-invariant, but this is **not** the full 7-value Hu invariant set	https://bioconductor.org/packages/EBImage	4.54.0 (release, Bioc 3.23) [doc5]; devel 4.55.2 built 2026-10-06 [doc6]	https://rdrr.io/bioc/EBImage/man/computeFeatures.html [doc7]

Momocs (archaeology-standard alternative)	Not Hu moments at all, but the de facto R package for 2D shape/outline analysis in archaeology (elliptical Fourier transforms, PCA on shape, classification) — worth knowing about even though it answers your research question differently	http://momx.github.io/Momocs/ ; CRAN: `install.packages("Momocs")`	Actively maintained but author notes it's being superseded by **Momocs2** (https://momx.github.io/Momocs2/, still pre-CRAN/GitHub-only) [doc8] [doc9] — I could not confirm an exact current version number for Momocs on CRAN today	http://momx.github.io/Momocs/ (homepage has a one-line full-pipeline example) [doc8]

My honest recommendation: given your beginner R level and the Kokentau dataset, Rvision + its `moments()`/`huInvariants()` pair is the only R route that gives you literal Hu moments, but it requires installing OpenCV as a system dependency (not trivial for a few-weeks-in R student) and isn't on CRAN. If that setup proves too heavy, EBImage's moment-based shape features are a legitimate, well-documented, CRAN/Bioconductor-installable fallback you can describe transparently in your methods as "moment invariants (center of mass, eccentricity, orientation) rather than the full Hu set." I'd flag this tradeoff to your instructor early rather than hit an installation wall late in the project.

[10/29/2026 01:47:33 +00:00] Jillyan M. Corrales:

how about Zernike moments? would that work better?

Response:

Short answer: Zernike moments are theoretically a stronger shape descriptor than Hu moments (they're orthogonal, so there's less redundant information, and they tend to discriminate shapes better) [doc1]. But for *your* specific situation — a beginner R user working in Quarto — they're actually **harder to implement**, not easier, because there is no mature, beginner-documented R package that computes shape-descriptor Zernike moments. I want to be upfront about that gap rather than paper over it.

The key caveat

There are two completely different things called "Zernike moments" in the software world, and it matters which one you find:

1. **Shape-descriptor Zernike moments** (what you want — rotation/scale-invariant numeric features describing a painted motif's outline or filled silhouette) [doc1]. This is implemented in Python's `mahotas.features.zernike_moments()` [doc2][doc1], which is the standard tool computer-vision tutorials use.
2. **Optical wavefront Zernike polynomials** (used in optics/astronomy to describe lens aberrations and telescope mirror errors) [doc3]. This is what the one real R package named `zernike` on GitHub actually

does — it's built for interferogram/wavefront analysis, not image shape description, and its README is written for people analyzing telescope mirrors, not archaeological motifs [doc3].

I could not find a CRAN or well-documented GitHub R package that implements shape-descriptor Zernike moments. That's a genuine gap, not something I'm glossing over.

Two short plain-language readings

1. **Mallick, S. & Bapat, K. (2018, updated 2026). "Shape Matching using Hu Moments" background section + cross-reference — see instead: 10.5 Zernike Moments, *Computer Vision* course notes blog.** https://cvexplained.wordpress.com/2020/07/21/10-5-zernike-moments/ — ~15–20 min [doc1]. This is the clearest plain-language explanation I found: it directly contrasts Zernike moments with Hu moments (which you already read about), explains orthogonality in one paragraph without heavy math, and gives honest pros/cons ("requires very precise segmentation," "only good for simple 2D shapes") that you can cite directly in a methods-limitations section.

2. **Tutorialspoint. "Mahotas – Zernike Moments."** https://www.tutorialspoint.com/mahotas/mahotas_zernike_moments.htm — ~10 min [doc2]. A short, concrete, syntax-first page showing exactly what `zernike_moments(image, radius, degree)` returns and how changing radius/degree changes the output vector — useful as a second, more mechanical companion to the first reading.

I was not able to verify a true "beginner textbook chapter" (under 20 pages) on Zernike moments the way I could for k-means with ISLR — this method just doesn't have that kind of intro-stats-textbook treatment. The original math-heavy source is Teague (1980), *J. Opt. Soc. Am.* 70(8):920–930, referenced in [doc1] and [doc4], but that is not beginner-accessible.

R tutorials — another honest gap

I could not find any R-specific tutorial (tidyverse-style or otherwise) that walks through computing shape Zernike moments in R, beginner or advanced. Every runnable example I found is in Python/OpenCV/mahotas:

1. https://cvexplained.wordpress.com/2020/07/21/10-5-zernike-moments/ — ~30–40 min to read and trace through the full worked Python example (shape matching a game cartridge against distractor objects) [doc1].
2. https://github.com/AyushAcharya07/Image_Feature_Extraction_Zernike_Moments — ~30 min [doc5]. A small, runnable Jupyter notebook project doing almost exactly your workflow (extract shapes → compute Zernike moments → compare). Again Python, not R, but it's the closest thing to a "beginner tutorial" that exists for this method, and the logic ports conceptually.

Three packages — reframed honestly

Package	Best for	Repo	Latest version/date	Walkthrough

mahotas (Python, not R)	The only mature, well-documented implementation of true shape-descriptor Zernike moments	https://github.com/luispedro/mahotas	I don't have a confidently verified current version/date — don't guess	https://www.tutorialspoint.com/mahotas/mahotas_zernike_moments.htm [doc2]
zernike (R, by mlpeck)	NOT shape description — optical wavefront/interferogram Zernike polynomial fitting; listed so you don't mistake it for what you need	https://github.com/mlpeck/zernike [doc3]	Actively maintained (178 commits), but no CRAN release date I can confirm — appears to be GitHub/Windows-binary only, not on CRAN [doc3]	README itself is the walkthrough: https://github.com/mlpeck/zernike/ [doc3]
shapes (R, CRAN, Dryden & Mardia)	A legitimate, CRAN, well-documented alternative for statistical shape analysis in R (landmark-based Procrustes analysis, PCA on shape) — doesn't compute Zernike moments, but is the closest *archaeology-appropriate*, beginner-installable R package for quantifying and clustering shape variation	https://cran.r-project.org/package=shapes	1.2.8, published 2025-10-24 [doc6]	Reference manual at the CRAN page above [doc6]; underlying method is documented in Dryden, I.L. & Mardia, K.V. (2016), *Statistical Shape Analysis with Applications in R*, 2nd ed., Wiley

My honest recommendation

Given you're a few weeks into R: computing real Zernike moments would require either (a) working in Python via `mahotas` and importing the resulting feature vectors into R as a CSV for your clustering step, or (b) using R's `shapes` package, which trades "Zernike moments" for landmark-based Procrustes shape analysis — a different but archaeologically well-established way to quantify motif shape that then feeds cleanly into k-means on shape-PC scores. I'd lean toward (b) for a project at your R skill level and timeline, and frame it in your writeup as "Zernike moments were considered but R tooling is immature; Procrustes-based shape descriptors via the `shapes` package were used instead as a comparable rotation/

scale-invariant alternative." That's a legitimate, defensible methodological choice to present to your instructor, and I'm happy to help you sketch that pipeline next.

[10/29/2026 01:52:31 +00:00] Jillyan M. Corrales:

how about contours for feature extraction?

Response:

Good news: contour-based feature extraction is actually the most mature, best-documented, and most beginner-friendly option you've asked about so far — especially because it has a dedicated R package built specifically for extracting shape outlines for morphometric analysis, which already has an archaeology-oriented audience.

Two short plain-language readings

1. **GeeksforGeeks. (2025, updated July 23). "What Are Contours in Computer Vision?"** https://www.geeksforgeeks.org/computer-vision/what-are-contours-in-computer-vision/ — ~10 min read [doc1]. This is the clearest plain-language definition I found: a contour is "a curve joining continuous points along the boundary that share color or intensity," and the page walks through properties (area, perimeter, orientation, convexity), detection methods, and limitations without requiring any code literacy — a good first stop before touching R.

2. **Malche, T. (2024, October 30). "What Is Image Contouring?" Roboflow Blog.** https://roboflow.com/blog/image-contouring — ~17 min read [doc2]. A step-by-step conceptual walkthrough (preprocess → binarize → detect → draw) with visuals explaining retrieval modes and approximation methods; useful as your second reading because it explains *why* each preprocessing step (grayscale, thresholding) matters, which maps directly onto the cleaning steps you'll need for rock-art photographs.

Two free beginner tutorials for contours in R

1. **R-bloggers / BNOSAC. (2020, August 3). "Finding contour lines."** https://www.r-bloggers.com/2020/08/finding-contour-lines/ — ~15–20 min [doc3]. A short, complete, copy-pasteable R example using `magick` to load an image, convert to grayscale, and run `image.ContourDetector::image_contour_detector()` to extract and plot contour lines — this is the most "beginner can run this in one sitting" R-native contour tutorial I could verify.

2. **imager package documentation: `contours()` function reference and "Getting started with imager" vignette.** https://rdrr.io/cran/imager/man/contours.html — ~15 min [doc4]. Not a long-form tutorial, but it's short, runnable, and tidyverse-adjacent (`purrr::walk()` is used directly in the example) to draw extracted contour lines over an image — a good second, lighter-weight approach to compare against the first.

(Caveat: neither is a long polished "tutorial" in the Datacamp/Datanovia sense — R's contour-detection tooling is covered mostly through package docs and short blog posts rather than full courses. I'm flagging that rather than inventing a longer tutorial that doesn't exist.)

Three best-documented, current R packages for contour-based feature extraction

Package	Best for	Repo	Latest version/date	Walkthrough
imager	General-purpose image processing with a built-in `contours()` function (wraps base R's `contourLines()`) plus `boundary()` for pixel-set edges — good first choice since it's CRAN-stable and tidyverse-friendly (works with `purrr`, suggests `dplyr`/`ggplot2`)	https://cran.r-project.org/package=imager	1.0.8, published 2025-12-23 [doc5] (a newer 1.0.9 build also appears on the maintainer's r-universe page, but I'd treat 1.0.8 as the confirmed CRAN release until you check directly) [doc6]	https://rdrr.io/cran/imager/man/contours.html [doc4], plus the "Getting started with imager" vignette linked from the CRAN page [doc5]

image.ContourDetector	A single-purpose package built exactly for this: the Unsupervised Smooth Contour Detection algorithm (von Gioi & Randall, 2016), giving you sub-pixel contour curves as a tidy `data.frame` (`x`, `y`, `curve` columns) — this output format is ideal for feeding straight into spatial clustering	https://github.com/bnosac/image (CRAN: https://cran.r-project.org/package=image.ContourDetector)	0.1.2, published 2025-11-27 [doc7][doc8]	The CRAN reference manual itself is a full runnable walkthrough: https://cran.r-project.org/web/packages/image.ContourDetector/refman/image.ContourDetector.html [doc8]
Momocs	The archaeology-standard package for turning extracted outlines into quantitative shape descriptors (it uses contour/outline detection internally via functions like `import_jpg()` and `Conte()`, then runs elliptical Fourier analysis and PCA on the shapes) — the natural next step after you've extracted contours, if your research question is really about *shape similarity* between motifs, not just their presence	http://momx.github.io/Momocs/ (CRAN: `install.packages("Momocs")`)	I could not confirm an exact current CRAN version/date with full confidence this session — the maintainers note active work is shifting to a successor package, **Momocs2** (https://momx.github.io/Momocs2/), still GitHub-only [doc14 — not separately verified this turn, carried from earlier research]	http://momx.github.io/Momocs/ homepage has a full one-page pipeline example from image import to shape PCA

Knowledge cutoff reminder: training data ends early 2025; all facts above were checked live today (October 2026), except the Momocs version note, which I'm explicitly flagging as unverified this session.

My honest take on fit for your project

Contours are a strong, defensible choice for the Kokentau dataset, and notably easier to get running in R than either Hu or Zernike moments were. My suggested pipeline: use `image.ContourDetector` or `imager::contours()` to extract each motif's boundary as x/y points, compute simple shape summaries

from those points (e.g., area, perimeter, centroid, bounding-box aspect ratio — all easy with base R or `sf`), then feed those numeric features plus the panel's spatial coordinates into k-means. That keeps your whole pipeline inside tools you already have some footing in (dplyr-style data frames) rather than requiring Python/OpenCV interop. Happy to help you draft that actual Quarto workflow next.